

Software, APIs, *and data.*

Follow a request from input to result.

Build the foundations around an AI model.

🕒 25-30 min core reading · Builds on Chapter 4 · 12 short sections

A useful AI application needs more than a model. It must receive inputs, move data between programs, handle failures, and keep records you can trust.

WHAT YOU WILL BE ABLE TO DO

Explain an API request, validate its data, diagnose common failures, and distinguish temporary memory from persistent storage. Run a local Python service, call it from another program, and query the saved predictions.

Reading path: Follow one runtime-prediction example from screen to storage. Then use the failure explorer and starter project. Optional notes add depth; allow extra time for the practical work.

1. Follow a request through an application

Imagine a small tool that estimates how long a batch job will take. A user enters its file size and receives an estimate. Several pieces cooperate:

The **frontend** displays the form and result. It runs in the user's browser in this example.

The **backend** receives the request, checks it, and coordinates the calculation. It runs as a separate program.

The **model or calculation** maps a valid input to a prediction.

A **database** stores records that need to outlive a single request.

➤ Separate the responsibilities

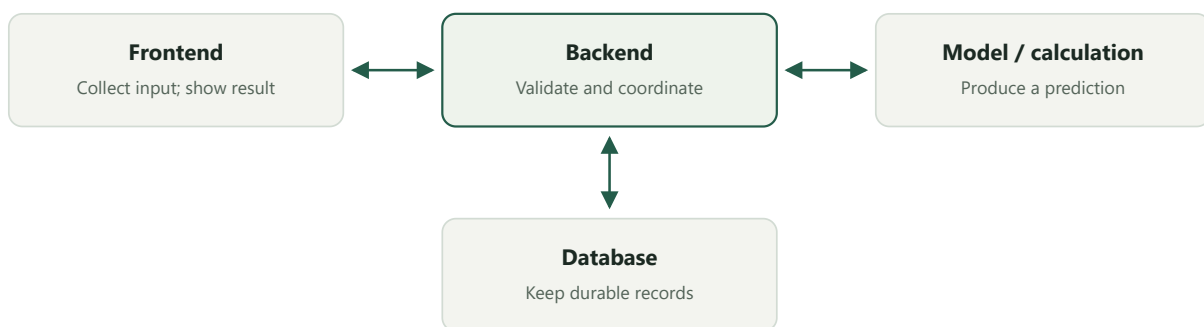


Figure 1. The backend coordinates prediction and storage. The database is not itself the model. Arrows show requests and returned results.

These are responsibilities, not necessarily four computers. A frontend and backend can both run on your laptop. In the starter, a Python client replaces the form, and that client saves the result locally.

The calculation remains Chapter 4's fixed formula: **predicted seconds = 2 × size in MB + 1**. For 3 MB, it returns 7 seconds. This is synthetic teaching code, not a trained or reliable runtime predictor. Later, a learned model can sit behind the same interface.

▼ Go deeper: library, framework, SDK, and service

A **library** provides reusable code that you call. A **framework** provides an application structure and calls your code at defined points. An **SDK** bundles tools or libraries for working with a platform, often including its API client. A **service** is a running program that other programs can contact. A service must be running before a client can contact it. Importing an SDK and calling a remote operation are separate steps.

2. Read an API request as a contract

An **API**, or application programming interface, defines how software can interact. A Python function has an interface too. A **web API** exposes operations over a network, often using HTTP, the request/response protocol used on the web.

The **client** sends a request; the **server** handles it and returns a response. An **endpoint** identifies an operation, such as `POST /predict`. Read this simplified HTTP/1.1-style message:

```
POST /predict HTTP/1.1
Host: 127.0.0.1:8765
Content-Type: application/json

{"size_mb": 3}
```

The **method** is POST. The **path** is `/predict`. **Headers** describe the message; Content-Type says the body contains JSON. The **body** carries the input. HTTP libraries add framing headers such as Content-Length for you.

In `http://127.0.0.1:8765/predict`, the scheme is HTTP, the host is your own computer's loopback address, and port 8765 selects the listening service. A remote API might use a domain name and **HTTPS**, which encrypts transport and authenticates the server through certificates. It does not establish that a prediction is correct.

GET usually retrieves a resource; **POST** submits data for processing or creation. PUT replaces a resource, PATCH changes part of it, and DELETE requests removal. Read the endpoint's actual contract rather than guessing behavior from its name. A query string, such as `/jobs?limit=20`, supplies URL parameters separately from a JSON body. See MDN's HTTP overview.

3. Separate JSON syntax from data meaning

Serialization converts in-memory values into a transferable representation. JSON encodes objects, arrays, strings, numbers, booleans, and null as text. **Parsing** turns that text back into values.

```
import json

body = json.dumps({"size_mb": 3}).encode("utf-8")
decoded = json.loads(body.decode("utf-8"))
assert decoded["size_mb"] == 3
```

UTF-8 is the text-to-bytes encoding used here. Python dictionaries are not wire messages: the request carries bytes. JSON uses double-quoted keys and strings, and `true`, `false`, `null` rather than Python's `True`, `False`, `None`. See Python's JSON reference.

A **schema** describes the expected structure and constraints. Our request needs an object with a numeric `size_mb` , from 0 to 1,000,000 inclusive. The response needs a numeric `prediction_seconds` and a model-version string.

Input	Valid JSON?	Valid for this request?
<code>{"size_mb": 3}</code>	Yes	Yes
<code>{"size_mb": "3"}</code>	Yes	No: string instead of number
<code>{"size_mb": -3}</code>	Yes	No: outside the accepted range
<code>{"size_gb": 3}</code>	Yes	No: the required field is missing

Validate at boundaries, where data enters or leaves your code. A response with status 200 and valid JSON can still have the wrong fields, units, or values. Even a valid schema cannot prove the model’s answer is accurate.

▼ Go deeper: missing, null, zero, and schema changes

A missing field was not supplied. JSON null explicitly represents no value. Zero is a number. Treating all three as zero can turn an unknown measurement into a false observation. Our API rejects missing or null size; zero is allowed.

A **schema version** identifies a data contract. A **model version** identifies the predictor used. They are different: replacing model weights may leave the response shape unchanged. Renaming a required field can break clients even if the model stays the same. Document changes, test old consumers, and use an explicit migration or compatible transition.

4. Treat failure as part of the interface

A response includes a **status code**, headers, and sometimes a body. Status codes describe request handling, not model accuracy.

Result	Meaning and useful next action
200	Request succeeded. Parse and validate the body before using it.
400	Bad request. Inspect the input and the endpoint’s error message.
401 / 403	Missing or invalid authentication / access refused. Check credentials and permissions.
404	Resource or route not found. Check the URL and identifier.
429	Too many requests. Follow the service’s limit policy and any Retry-After guidance.
500 / 503	Server error / service unavailable. A temporary problem is possible; investigate before retrying.

A **timeout** means the client stopped waiting during an operation. It is not an HTTP status, and it does not prove the server did nothing. A **retry** sends another attempt. Retry only when the failure and operation make that appropriate; cap attempts, increase the delay between them (**backoff**), and add a small random variation (**jitter**) to avoid synchronized bursts. See HTTP status meanings and 429 handling.

▼ Go deeper: idempotency and duplicate work

An operation is **idempotent** when repeating the same request has the same intended server effect as doing it once. Responses and logs can still differ. POST is not automatically idempotent: retrying “create a report” may create or bill for two reports if the first response was lost.

An API may support an **idempotency key**: reuse a unique identifier for retries of one operation, according to that API’s documented rules. Sending a made-up header does nothing unless the server implements it. Our prediction server creates no records, but the client appends a database row each time it completes successfully. Running it twice creates two history rows. See [HTTP method semantics](#).

5. Diagnose a request before changing the model

FIND THE FAILED BOUNDARY

Local simulation

Same task. Different outcomes.

Choose a situation and predict your next action. These examples run inside the page; no server is contacted and no credentials are needed.

Example request: POST /predict with JSON body `{"size_mb": 3}`. A successful response contains `prediction_seconds = 7` and `model_version = demo-v1`.

Situation	Outcome	Reasoned next action
Valid request	200 OK	Validate the fields and units, then use or save the result. Accuracy still needs comparison with observations.
Negative size	400 Bad Request	Correct the input. Repeating the same negative size does not fix it.
Missing credentials	401 Unauthorized	Check credentials on the backend. This simulated protected endpoint differs from the starter, which has no authentication.
Rate limit	429 Too Many Requests	This example includes <code>Retry-After: 2</code> , meaning wait at least two seconds. Follow the API policy and use bounded retries only when appropriate.
No response in time	No HTTP response	The client stopped waiting. The server may have completed the work; check the operation outcome and duplicate-work risk.
Unexpected response shape	200 OK	Do not silently treat the missing <code>prediction_seconds</code> field as zero. Reject the result and investigate the API version or schema mismatch.

Use the explorer to locate the failure: input, access, availability, transport, or response contract. A prompt change cannot repair an invalid credential. More training cannot fix a wrong endpoint. First find which boundary failed.

Authentication establishes an identity or credential; **authorization** decides what that identity may access. A service API key often grants access under an account. Keep provider secrets on the backend, outside browser code and version control. Load them from protected runtime configuration or a secret store; avoid printing them in logs. Our local lesson needs no credentials.

▼ Go deeper: [waiting, async work, streaming, and CORS](#)

In a **synchronous** call, the caller waits for completion. **Asynchronous** code can let other work proceed while waiting; it does not make the model's computation intrinsically faster. A long job may return a job ID so the client can check its status later.

Streaming delivers a response incrementally. Displaying early text is useful, but those chunks may be incomplete; wait for completion before treating them as a finished structured result.

CORS is a browser mechanism controlling whether a page can read responses from a different origin. An origin includes scheme, host, and port. A Python client is not subject to browser CORS checks. CORS is not authentication: it does not stop arbitrary non-browser clients calling an API. See [MDN's CORS guide](#).

6. Choose how to represent and store data

Memory holds working values while a process runs; stopping the process usually loses them. **Persistence** keeps data beyond that process, for example in a file or database.

Form	A useful role	Limitation to remember
CSV	Flat rows for import or inspection	Types, missing values, and quoting need agreed rules
JSON	Nested API messages and configuration	Structure alone supplies neither a schema nor data quality
Relational database	Tables with queries, constraints, and transactions	Design the fields and relationships deliberately
Object storage	Files such as source PDFs, images, or dataset snapshots	Querying records usually needs another layer

In a relational table, a **row** is a record and a **column** is a field. A **primary key** identifies a row. A **foreign key** references a key in another table, linking records. An **index** can accelerate particular lookups, at the cost of storage and write work.

An **API** describes how to interact; a **database** stores and queries data. They are not alternatives. A backend can expose an API while reading its database internally. Also, persisting a conversation does not train a model or ensure that the next request will include that conversation.

▼ Go deeper: relational, document, vector, and cache storage

A document database organizes records as documents, often with nested fields. A vector index supports similarity search over numeric representations called embeddings; we will build those in the search chapters. Some databases combine these capabilities. Similarity does not prove truth or grant access to a document.

A **cache** keeps reusable results to reduce repeated work. Its values can become stale, so decide when to refresh or invalidate them. Include relevant model, input, and access context in a cache design; a result for one user may be inappropriate for another.

7. Ask precise questions with SQL

SQL is a language for working with relational data. Suppose the starter has saved these prediction records:

id	size_mb	prediction_seconds	model_version
1	1.0	3.0	demo-v1
2	2.0	5.0	demo-v1
3	3.0	7.0	demo-v1

```
SELECT id, size_mb, prediction_seconds
FROM predictions
WHERE prediction_seconds >= 5
ORDER BY id;
```

SELECT chooses columns, FROM names the table, WHERE filters rows, and ORDER BY specifies order. The result contains IDs **2 and 3**. Without ORDER BY, do not rely on the returned order. A **JOIN** combines matching rows from tables; an **aggregation**, such as AVG, summarizes a group. See the [SQLite SELECT reference](#).

When a value comes from a user, bind it separately instead of building SQL with string concatenation. In Python's sqlite3 API, `?` marks a value to bind:

```
rows = connection.execute(
    "SELECT id FROM predictions WHERE prediction_seconds >= ? ORDER BY id",
    (5.0,),
).fetchall()
```

Here `connection` is an open database connection. `(5.0,)` is a one-item tuple, and `fetchall()` collects the result rows. This fragment assumes the table already exists; the starter provides the runnable setup.

▼ Go deeper: transactions and trustworthy records

A **transaction** groups database changes into a unit that commits or rolls back. It prevents a partially completed group of writes from being accepted as a completed change. It does not make an external API call and a database write one automatic transaction.

The starter commits one inserted row after validating the API response and closes its connection. If the API succeeds but saving fails, the client reports that failure. These stored values are predictions, not measured runtimes: training labels would require separately collected observations. See [Python's sqlite3 guide](#).

8. Make the data path reproducible

A **data pipeline** is a sequence of steps that moves and transforms data. **ETL** means extract, transform, load; **ELT** loads before transformation. For a small prediction history, consider this path:

➤ Make each transformation explainable



Figure 2. Preserve source context, check meaning as well as format, and keep the accepted records traceable to their origin.

Keep the original source available when appropriate, then check required fields, units, allowed ranges, missing values, and duplicates. Record what was rejected and why. Converting 2 GB to 2 MB is a unit error that perfectly valid JSON will not catch. Removing duplicates requires a meaningful identity; two similar jobs may be separate events.

Provenance records where data came from and how it changed. For an AI result, keep enough context to reproduce or investigate it: an input reference, code and model versions, relevant settings, and timing. Log status and latency without needlessly copying sensitive inputs or credentials.

Use **Git** to version code and small configuration files; a **commit** is a recorded snapshot. Review changes with a diff. Keep secrets and generated databases out of commits. Record dataset versions separately when datasets are too large or unsuitable for the code repository. See [Git's version-control introduction](#).

A **unit test** checks a small piece, such as the prediction function. An **integration test** checks cooperating pieces, such as sending an actual request to the service and reading its response. A **model evaluation** measures prediction quality on suitable examples. Passing software tests does not establish model quality.

9. Run a local API and save its result

Extract the starter into a folder. It contains **server.py**, **client.py**, **inspect_data.py**, and a README. Use Python 3.10 or newer; no third-party packages are needed. Open two terminals in that folder. Replace `py` with your working Python command if necessary.

In terminal A, start the service and leave it running:

```
py server.py
```

In terminal B, make a request and inspect the saved result:

```
py client.py 3
py inspect_data.py
```

The first command reports **Prediction: 7.0 seconds (demo-v1)** and saves a row in `predictions.db` beside `client.py`. A fresh database starts at row 1. The inspection script queries predictions of at least 5 seconds, so it displays this row. Later successful client runs append records; inspection only reads them.

➤ An actual local request, then a durable record



Figure 3. The Python client calls the service and validates the returned JSON. Only then does the client save a history row; the server keeps no prediction history.

Follow these responsibilities in the code:

server.py parses the request, validates size, and returns the fixed formula's result as JSON.

client.py encodes the input, calls `/predict` with a timeout, validates the response, then inserts one SQLite row using bound values.

inspect_data.py opens the existing database for reading and runs the filter query.

The client sets a five-second timeout on blocking network operations and makes one attempt. That setting is not a strict deadline for the entire program. Stop the server with **Ctrl+C**. If port 8765 is occupied, use `py server.py -port 8766` and `py client.py 3 -port 8766`.

What this project establishes: two real Python programs communicate over local HTTP and store a checked result. The fixed formula, accepted input range, and successful response do not establish prediction accuracy. Python's built-in HTTP server is a learning tool here; it is bound to your computer's loopback address and is not a production deployment.

▼ Read the request-building code

```
import json
from urllib.request import Request, urlopen

body = json.dumps({"size_mb": 3}).encode("utf-8")
request = Request(
    "http://127.0.0.1:8765/predict",
    data=body,
    headers={"Content-Type": "application/json"},
    method="POST",
)
with urlopen(request, timeout=5) as response:
    result = json.loads(response.read().decode("utf-8"))
print(result["prediction_seconds"]) # 7.0
```

Run this only while the server is active. `Request` constructs the message; `urlopen` sends it. The `with` block closes the response when finished. This short fragment shows the mechanics; the downloadable client also checks the content type, status, and fields, reports failures, and persists successful results. See [urllib.request](#) and the [HTTP server documentation](#).

10. Practice: predict, break, inspect

Exercise 1: change the input

With the server running, predict the output of `py client.py 4`, then run it. What changes in the database?

▼ Reveal the result

The formula gives $2 \times 4 + 1 = 9$ seconds. The client saves one new row with `size_mb = 4` and `prediction_seconds = 9`. The earlier row remains. No weights were learned or adjusted.

Exercise 2: distinguish two failures

Run `py client.py -1` while the server is active. Then stop the server and run `py client.py 3`. Should either attempt create a history row?

▼ Reveal the diagnosis

The negative size receives HTTP 400: the server was reached but rejected the input. With the server stopped, the connection fails, usually before any HTTP response exists. Neither attempt passes the client's successful-response checks, so neither creates a row. Retrying the negative input unchanged will not repair it.

Exercise 3: separate a prediction from evidence

A response has status 200 and a schema-valid prediction of 7 seconds. The actual job takes 11 seconds. Did HTTP fail? What measurement is missing from the starter's stored data?

▼ Reveal the distinction

The API call succeeded; its prediction was inaccurate by 4 seconds. Store the separately measured actual runtime and the matching job identifier if you want to evaluate such errors. A prediction history alone cannot tell you which predictions were correct.

11. Check your understanding

Answer all five questions, then read the explanations.

01 An API returns 200 and valid JSON. What should you conclude?

- A. The prediction is accurate.
- B. The request succeeded; validate the response contract and evaluate accuracy separately.
- C. The data can immediately be used as a training label.

Answer: B. HTTP status describes handling of the request. Data validation and model evaluation answer different questions.

02 The contract requires a numeric `size_mb`. Is `{"size_mb": "3"}` acceptable?

- A. No. It is valid JSON, but the field is a string.
- B. Yes. Every valid JSON object follows the contract.
- C. Yes. Quotation marks do not affect types.

Answer: A. Parsing checks syntax. Schema validation checks required fields, types, and constraints. This contract does not accept a numeric string.

03 A create-report request times out. Is immediately repeating it always safe?

- A. Yes. A timeout proves the server stopped.
- B. Yes. POST is always idempotent.
- C. No. The first attempt may have completed; check retry and idempotency rules.

Answer: C. A lost response leaves the server outcome uncertain. An API-supported idempotency mechanism can prevent duplicate intended effects.

04 Where should a provider API secret used by a web app normally live?

- A. In the JavaScript downloaded to every browser.
- B. In protected backend configuration, outside version control.
- C. In a public screenshot of the API request.

Answer: B. Users can inspect browser-delivered code. The backend should load protected credentials and enforce access rules without exposing the provider secret.

05 The client runs successfully twice with the same size. What does this starter do?

- A. Appends two prediction-history rows; it does not retrain the formula.
- B. Trains the model twice.
- C. Always keeps only one row because POST removes duplicates.

Answer: A. Each successful client run inserts a row. Stored predictions are neither observed targets nor automatic training updates.

12. Connect this to your future AI applications

For an LLM app, the request may contain messages and the response may contain generated text. For RAG, the backend also retrieves relevant records before calling the model. The same boundaries remain: request, access, data contract, processing, response validation, and persistence. We will build those specialized parts in later chapters.

▼ Quick-reference glossary

Term	Recall cue
Client / server	Sends a request / handles it
Endpoint	An API operation identified by its method and location
Header / body	Message metadata / message payload
Serialization / parsing	Values to a transferable representation / representation back to values
Schema	Expected structure, types, and constraints
Authentication / authorization	Establish identity or credentials / decide allowed access
Timeout / retry	Stop waiting / make another attempt
Persistence / cache	Keep data beyond a process / reuse results with a freshness policy
Transaction	Changes committed or rolled back as a unit
Provenance	Data origin and transformation history

The examples and diagrams are original and the data is synthetic. Official references are linked beside the relevant topics. The starter uses Python's standard library; the website explorer simulates responses locally.

Next: Chapter 6: Algebra and linear algebra for AI.